Module 01 | Introduction to Deep Learning and Perceptrons

My Solution to Practice Problems

Table of Contents

Bias

What is a bias in a perceptron?

The `Bias` (b) is a parameter of any standard `AN` , not only of a perceptron, additional to the `Weights` $(W = [w_i])$ parameter, that the perceptron learns tentatively during training, and uses it, together with `Weights` , during both training and prediction, to combine an input observation linearly toward the output.

Why do we add bias to a perceptron?

Let's recall a key step in how a trained perception predicts. It starts with `combining the inputs linearly` . The better the `Linear Combination` , the better the prediction performance. Well, then what decides a linear combination is good or bad, and how is a perceptron supposed to have it or acquire it?

A perception learns the optimal linear combination parameters (`Weights and Bias`) from training.

What ensures optimality? The perceptron learns the parameters, during training, with supervision of truth values and by optimizing a loss function, called 'Perceptron Loss', that is tied to its activation function, namely, the step function.

This Linear Combination Parameters (`Weights and Bias`) represent a 'Linear Decision Boundary' on the feature space. This decision boundary is a line on 2D, a plane on 3D (generally, a hyperplane on any dimensional feature space).

From coordinate geometry, not from ML, not from DL, from Coordinate Geometry, this dicision boundary has a equation of locus, with Feature values as coordinate variables, and Weights and Bias as equation parameters.

From Coordinate Geometry, the `general equation of n-dimensional hyperplane` is this:

$$w_1x_1 + w_2x_2 + \cdots\cdots\cdots + w_nx_n + b = 0$$

If the decision boundary passes through the coordinate origin, this $\text{Bias} = b = 0$, and we can omit the term, in other wors, no bias added.

If the decision boundary does not pass through the coordinate origin, the bias is not zero. It must be there as put by Coordinate Geometry, again not by ML, not by DL.

We add the `Bias` term while predicting with a perceptron, neither because we want it for mere engineering convenience, nor it's an ML/DL adaptation of the underlying Mathematics. It is already in the perception, or not, decided during training depending the distribution of the feature space, to be added (if there) with $\sum w_i x_i$ during prediction.

Weights

What are weights in a perceptron?

`Weights` $(W = [w_i])$ are the measures of importance of the individual features/inputs toward the final decision or output of the perceptron.

How do weights affect the output of a perceptron?

Suppose, from training, the Perceptron learns that the weight of input x_i is w_i.

$$\text{Influence of a Feature on Activation Decision: } \begin{cases} \text{None}, & \text{if } w_i \text{ close to } 0 \\ \text{Direct}, & \text{if } w_i > 0, \text{ and the influence gets stronger as } |w_i| \text{ gets larger.} \\ \text{Inverse}, & \text{if } w_i < 0, \text{ and the influence gets stronger as } |w_i| \text{ gets larger.} \end{cases}$$

Perceptron Output

A perceptron has inputs $x_1 = 2$, $x_2 = 3$, weights $w_1 = 0.5$, $w_2 = 1$, and bias $b = 1$. Calculate the weighted sum (before activation).

We have,

$$\begin{aligned} X &= [x_1 \quad x_2] \\ &= [2 \quad 3] \end{aligned}$$

$$\begin{aligned} W &= [w_1 \quad w_2] \\ &= [0.5 \quad 1] \end{aligned}$$

$$b = 1$$

$$\begin{aligned} W \cdot X + b &= [0.5 \quad 1] \cdot [2 \quad 3] + 1 \\ &= 1 + 3 + 1 \\ &= 5 \end{aligned}$$

So, the weighted sum is 5.

Step function

What is the step function and how does it work?

The Step Functions, as defined below, is an activation function used in a standard perceptron.

$$y_{step}(x) = \begin{cases} 1, & x \geq \mathbf{0} \\ 0, & x < \mathbf{0} \end{cases}$$

This $\mathbf{0}$ is called the `(Decision) Threshold` of the step function.

When this function is used as the activation function,

$$x = \text{Weighted Sum}.$$

That is, the linear `Weighted Sum` is the input to the step function in a perceptron, as to the activation function of a neuron of any other kind.

The step function, as the activation function of a perceptron, decides on the sign of the linear `weighted sum` .

If the `Weigted Sum` is greater than or equal to `0` , that is, `positive in sign` , the perceptron's output is `1` , that is, the perceptron fires, or gets activated.

If the `Weigted Sum` is less than `0` , that is, `negative in sign` , the perceptron's output is `0` , that is, the perceptron does not fire, or decides not to get activated.

Activation Function

Why do we need an activation function in a perceptron?

Not only a perceptron, but any neuron of any kind needs an activation function.

An standard `AN` first combines the feature values of an input example with `Weights and Bias` into a `(Linear) Weighted Sum`.

Then this linear `Weighted Sum` is fed to the activation function, which generate the (final) output of the AN.

An activation function is an opportunity, with endless possibilities, to modify the output of the `Weighted Sum` function before the neuron outputs.

Without an activation function, the `Weighted Sum` is the (final) output of the AN. This case is equivalent to using the identity function $f(x) = x$ as the activation function.

A beautiful and widely-accepted opportunity offered by Activation Function is the `opportunity to introduce non-linearity, into an AN, upon the Linear Weighted Sum`.

A perceptron, using the Step Function as its Activation Function, does not use this opportunity of `Non-Linearity'. It uses this opportunity to become a Binary Classifier, upon the Linear Weighted Sum Function $WS_{\vec{W},b}(\vec{X}) \equiv \vec{W} \cdot \vec{X} + b$, with $WS_{\vec{W},b}(\vec{X}) = 0$ as its decision boundary, succeeding only on Linearly Separable Datasets. Also, as the range of the step function is not continuous, but discrete with hard steps, it lacks probabilistic classification.

So, why does an standard AN need an activation function? To modify the output of the Linear Weighted Sum Function, before the AN outputs.

Why does a perceptron, which is a Binary Classifier, need the step function, that is, its activation function? To become a classifer. To decide on class of the input example. The decision is on the sign of the Linear Weighted Sum, that is, on the comparison between the Linear Weighted Sum and `0`, while the later acts as the Decision Threshold.

'Firing' Perspective: a perceptron needs the step function as its activation function to decide if the input values are, collectively, `important enough` for the perceptron to fire. The `importance` is represented by the `Weighted Sum`, which is the input to the step function. And the step function decides, comparing the `importance` with `Decision Threshold 0`, if the importance is `enough`.

Name three common activation functions.

1. Sigmoid
2. tanh
3. ReLU